Day 2 — LlamaIndex End to End

1. Core idea in simple words

LlamaIndex is a framework for building AI applications that can use your own data. It is not an LLM model. It is not a vector database. It is a toolkit that helps you connect an LLM to documents, databases, APIs, PDFs, slide decks, tickets, logs, and other business data.

The simplest mental model:

```
LLM alone = smart person with general knowledge

LlamaIndex + LLM = smart person with access to your company library
```

For a Disney-like Staff AI Engineer role, this matters because many backend AI systems are not only "chat with GPT." They need to answer from **private, fresh, permission-controlled, production data**: park operations manuals, content metadata, support policies, legal guidelines, internal runbooks, marketing docs, ride incident SOPs, booking rules, and more.

LlamaIndex currently positions itself as a framework for **context-augmented LLM applications**, with tools for data connectors, indexes, query engines, chat engines, agents, workflows, observability, and evaluation. (LlamaIndex Python Documentation)

---

2. Foundational concepts

What problem does LlamaIndex solve?

LLMs are trained on public or licensed data, but they usually do **not** know your company's private documents, latest APIs, internal policies, or database state.

So the problem is:

```
User asks a business-specific question
        ↓
LLM does not know the private/current answer
        ↓
Need to fetch relevant company data
        ↓
Give that data to the LLM
        ↓
LLM answers using that data
```

This is called **context augmentation**.

Context means information given to the LLM at question time.

Augmentation means adding extra useful information.

So **context augmentation** means: "Before asking the LLM to answer, give it the right company data."

LlamaIndex helps with this full pipeline: ingesting data, parsing it, indexing it, retrieving the right parts, and generating an answer.

What is RAG?

RAG means **Retrieval-Augmented Generation**.

Define each word:

Retrieval means finding useful information.

Augmented means improved by adding something.

Generation means the LLM writes an answer.

So RAG means:

```
Find relevant data first, then ask the LLM to answer using that data.
```

LlamaIndex docs describe RAG as a core technique for data-backed LLM apps: instead of training the LLM on private data, relevant parts of the data are provided to the LLM at query time. (LlamaIndex Python Documentation)

LlamaIndex vs plain Vanilla RAG

Vanilla RAG is the design pattern.

You manually build:

```
Load documents
→ chunk documents
→ create embeddings
→ store vectors
→ retrieve top chunks
→ send chunks to LLM
→ generate answer
```

LlamaIndex is a framework that gives you reusable building blocks for doing this.

Plain Vanilla RAG is like building a restaurant kitchen from raw parts.

LlamaIndex is like using a professional kitchen setup: counters, storage, tools, workflows, and safety checks are already organized.

You can still customize deeply, but you do not start from zero.

LlamaIndex vs LangChain

At a practical level:

LlamaIndex is strongest when your main problem is data.

Use it when the core challenge is:

```
How do I ingest, index, retrieve, cite, evaluate, and query my documents/
data well?
```

LangChain is more general-purpose for LLM app and agent integration. LangChain's current docs describe it as an open-source framework with prebuilt agent architecture and integrations for models and tools. (Docs by LangChain)

Simple comparison:

```
LlamaIndex:
Best mental model = data/retrieval framework

LangChain:
Best mental model = LLM app / tool / agent integration framework

LangGraph:
Best mental model = stateful workflow orchestration runtime
```

In a Disney backend system, you might use all three:

```
LlamaIndex → search internal content/policy/document knowledge
LangChain → connect models/tools/providers
LangGraph → control long-running multi-step workflows
```

Core mental model

Think of LlamaIndex as six connected steps:

```
1. Data ingestion
   Bring data into the system

2. Indexing
   Organize the data for search

3. Retrieval
   Find the most relevant pieces

4. Query engine
   Combine retrieval + LLM answer generation

5. Workflows
   Create multi-step AI processes

6. Agents
   Let an LLM choose tools/actions when needed
```

When LlamaIndex is a good fit

Use LlamaIndex when:

You have many documents, PDFs, web pages, API outputs, database rows, or support articles.

You need RAG with citations.

You need metadata filtering, such as:

```
Only retrieve docs where:
region = "US"
business_unit = "Disney Parks"
document_type = "policy"
effective_date <= today
```

You need ingestion pipelines, refresh logic, document updates, evaluation, or production retrieval tuning.

When LlamaIndex may not be a good fit

It may be unnecessary when:

You only need a simple one-off LLM prompt.

Your data is tiny and can fit directly into the prompt.

You need very custom low-level infrastructure and do not want framework abstractions.

Your main problem is not retrieval, but complex stateful orchestration. In that case, LangGraph or custom workflow code may be more central.

---

3. LlamaIndex building blocks

3.1 Document

A **Document** is a container for source data.

Example:

```
A PDF policy file
A Confluence page
A database row
An API response
A Markdown file
A support ticket
```

In LlamaIndex, a Document stores text plus useful attributes like metadata and relationships. (LlamaIndex Python Documentation)

Example:

```
Document:
text = "Guests can modify park reservations up to..."
metadata = {
  "source": "reservation_policy.pdf",
  "department": "Parks",
  "region": "US",
  "version": "2026-07"
}
```

3.2 Node

A **Node** is a smaller piece of a document.

Large documents are too big to send fully to the LLM. So we split them.

```
Document = full 40-page policy PDF
Node = one useful chunk from that PDF
```

LlamaIndex treats Nodes as first-class objects. A Node can contain text, metadata, and relationships to other nodes. (LlamaIndex Python Documentation)

This is important because retrieval usually happens at the **chunk/node level**, not the full-document level.

3.3 Chunk

A **chunk** is a small section of text.

Example:

```
Full document:
"Disney Park Reservation Policy..."

Chunks:
Chunk 1: Overview
Chunk 2: Cancellation rules
Chunk 3: Refund exceptions
Chunk 4: Annual pass rules
```

Good chunks are extremely important.

Bad chunking causes bad retrieval.

Bad retrieval causes bad answers.

3.4 Metadata

Metadata means extra labels about the data.

Example:

```
text = "Guests can cancel 24 hours before arrival."

metadata = {
  "product": "park_ticket",
  "region": "US",
  "policy_type": "cancellation",
  "effective_date": "2026-01-01",
  "access_level": "internal"
}
```

Metadata helps with filtering.

Without metadata, the retriever may search everything.

With metadata, the retriever can search only the correct subset.

For production systems, metadata is not optional. It is one of the biggest quality levers.

3.5 Loader / connector

A **loader** or **connector** brings data from a source into LlamaIndex.

Examples:

```
Local folder loader
Google Drive connector
S3 connector
SQL database reader
Notion connector
Slack connector
Web page reader
PDF parser
```

LlamaIndex docs describe data connectors as tools that ingest existing data from native sources and formats such as APIs, PDFs, SQL, and more. (LlamaIndex Python Documentation)

In backend terms, a connector is like an ETL input adapter.

3.6 Parser

A **parser** converts raw files into usable text or structured content.

Example:

```
PDF file
→ extract text
→ preserve sections
→ detect tables
→ detect headings
→ produce clean text
```

Parsing quality matters a lot.

A bad PDF parser may produce:

```
Refund is allowed not before cancellation guest policy...
```

A good parser produces:

```
Refunds are allowed only if the guest cancels at least 24 hours before
arrival.
```

The second version is much easier for retrieval and LLM answering.

3.7 Transformation

A **transformation** is a processing step applied during ingestion.

Examples:

```
Split text into chunks
Extract title
Extract summary
Extract metadata
Create embedding
Clean text
Remove boilerplate
```

LlamaIndex has an `IngestionPipeline` concept where transformations are applied to input data, and resulting nodes can be returned or inserted into a vector database. (LlamaIndex Python Documentation)

3.8 Embedding

An **embedding** is a list of numbers that represents meaning.

Example:

```
"How do I cancel a ticket?"
→ [0.12, −0.44, 0.87, ...]
```

Similar meanings have similar vectors.

So these two are close in vector space:

```
"How do I cancel a ticket?"
"Can a guest cancel park admission?"
```

But this is far away:

```
"What food is available in Epcot?"
```

Embeddings power semantic search.

3.9 Vector index

A **vector index** organizes embeddings so we can search quickly.

Without an index:

```
Compare query against every chunk one by one
```

With an index:

```
Quickly find nearby chunks
```

LlamaIndex's `VectorStoreIndex` is a common RAG building block. Its docs explain that vector stores accept Node objects and build an index from them, and that `from_documents` splits documents into Node objects that track metadata and relationships. (LlamaIndex Python Documentation)

3.10 Vector store

A **vector store** is a database for embeddings.

Examples:

```
Qdrant
Pinecone
Weaviate
Milvus
pgvector/Postgres
OpenSearch
Elasticsearch
Chroma
Redis
```

LlamaIndex supports many vector store integrations, and the docs list feature support such as metadata filtering, hybrid search, delete support, document storage, and async support across different vector stores. (LlamaIndex Python Documentation)

3.11 Retriever

A **retriever** finds relevant nodes for a user query.

Example:

```
User query:
"What is the cancellation rule for Disney park tickets?"

Retriever returns:
Node 17: cancellation policy
Node 31: refund exception
Node 44: annual pass cancellation note
```

The retriever does not write the final answer. It only finds evidence.

3.12 Query engine

A **query engine** is the full question-answering flow.

It usually does:

```
Take user question
→ retrieve nodes
→ send nodes + question to LLM
→ synthesize answer
→ return answer
```

LlamaIndex query engine composition can be customized at a lower level by explicitly creating a retriever, a response synthesizer, and a `RetrieverQueryEngine`. (LlamaIndex Python Documentation)

3.13 Response synthesizer

A **response synthesizer** takes retrieved chunks and produces the final answer.

Simple idea:

```
Retrieved chunks = raw evidence
Response synthesizer = answer writer
```

It decides how to combine multiple chunks into one answer.

Example:

```
Chunk A says cancellation is allowed 24 hours before.
Chunk B says special event tickets are non-refundable.
Chunk C says annual pass reservations have a different rule.

Synthesized answer:
"For normal park tickets, cancellation is allowed 24 hours before arrival.
Special event tickets may be non-refundable. Annual pass reservations
follow a separate rule."
```

3.14 Reranker

A **reranker** reorders retrieved results after the first retrieval.

Why?

The first retriever may return 20 possible chunks.

The reranker asks:

```
Which 5 chunks are truly most useful for this exact question?
```

This improves precision.

3.15 Workflow

A **workflow** is a multi-step process.

Example:

```
Step 1: classify question
Step 2: choose data source
Step 3: retrieve documents
Step 4: check policy version
Step 5: generate answer
Step 6: validate citations
Step 7: return response
```

LlamaIndex docs describe workflows as event-driven abstractions for orchestrating steps and LLM calls. (LlamaIndex Python Documentation)

3.16 Agent

An **agent** is an LLM-powered decision-maker.

It can choose tools, break down tasks, plan, remember previous work, and decide what to do next.

LlamaIndex docs describe agents as automated reasoning and decision engines that can break complex questions into smaller ones, choose tools, plan tasks, and use memory. (LlamaIndex Python Documentation)

Use agents carefully. They are powerful, but less predictable than deterministic code.

---

4. End-to-end flow

There are two main flows:

```
A. Ingestion-time flow
B. Query-time flow
```

A. Ingestion-time flow

This happens before the user asks a question.

```
Raw data
  ↓
Load documents
  ↓
Parse and clean
  ↓
Split into nodes/chunks
  ↓
Attach metadata
  ↓
Create embeddings
  ↓
Store in vector database
  ↓
Ready for retrieval
```

Disney example:

```
Input:
- Park ticket policy PDFs
- Guest support FAQs
- Internal escalation SOPs
- Reservation database export

LlamaIndex ingestion:
- Reads files
- Parses documents
- Splits into chunks
- Adds metadata like region, department, policy version
- Embeds chunks
- Stores them in Qdrant/Postgres/Pinecone/OpenSearch
```

B. Query-time flow

This happens when the user asks a question.

```
User question
  ↓
Convert question into embedding
  ↓
Retrieve similar chunks
  ↓
Apply metadata filters
  ↓
Optional reranking
  ↓
Send best chunks to LLM
  ↓
Generate grounded answer
  ↓
Return answer with citations
```

Disney example:

```
User:
"Can a guest cancel a park reservation on the same day?"

System:
1. Search only Parks policy docs
2. Filter region = US
3. Retrieve cancellation-related chunks
4. Rerank best chunks
5. Ask LLM to answer only from retrieved policy
6. Return answer with source citations
```

Minimal LlamaIndex-style code shape

This is the beginner mental model:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader("data").load_data()

index = VectorStoreIndex.from_documents(documents)

query_engine = index.as_query_engine()

response = query_engine.query("What is the cancellation policy?")

print(response)
```

The official quickstart shows the same high-level shape: load documents, create `VectorStoreIndex`, convert it to a query engine, query it, and print the response. (LlamaIndex Python Documentation)

In production, you would not stop here. You would customize parsing, chunking, metadata, vector store, filters, reranking, access control, evaluation, logging, and caching.

---

5. Inter-relation between ingestion, embeddings, retrieval, and response

This is the most important mental model.

```
Ingestion quality controls chunk quality.

Chunk quality controls embedding quality.

Embedding quality controls retrieval quality.

Retrieval quality controls context quality.

Context quality controls answer quality.
```

So the chain is:

```
Bad ingestion
→ bad chunks
→ bad embeddings
→ wrong retrieval
→ wrong context
→ hallucinated or weak answer
```

A Staff Engineer should not only say:

```
"Use better model."
```

A Staff Engineer asks:

```
Did we parse the document correctly?
Did we chunk by section boundaries?
Did we preserve metadata?
Did we retrieve the right policy version?
Did we filter by user permissions?
Did we evaluate retrieval separately from answer generation?
```

That is the production mindset.

---

6. Data ingestion foundations

Structured vs unstructured data

Structured data has a fixed shape.

Example:

```
SQL table:
ticket_id | guest_id | park | visit_date | status
```

Unstructured data does not have a fixed shape.

Example:

```
PDFs
Emails
Chat transcripts
Markdown files
Support articles
Legal documents
```

Semi-structured data is in between.

Example:

```
JSON
HTML
YAML
XML
Logs
```

LlamaIndex is especially useful when you need to connect LLMs to unstructured and semi-structured data.

Document pipeline

A **document pipeline** is the repeatable process that turns raw files into searchable knowledge.

Production pipeline:

```
Fetch source files
→ detect file type
→ parse content
→ clean text
→ split into chunks
→ extract metadata
→ validate metadata
→ embed chunks
→ upsert into vector DB
→ record ingestion status
→ monitor failures
```

In LlamaIndex, `IngestionPipeline` supports transformations, caching, async operation, document management, and parallel processing. Its document management can use document IDs and hashes to detect duplicates or changed documents. (LlamaIndex Python Documentation)

Why ingestion quality matters

Imagine a Disney support bot answering policy questions.

If the PDF parser loses table structure, the system may confuse:

```
Refund allowed: Yes
Refund allowed: No
```

If metadata is missing, the system may answer from the wrong region:

```
US policy vs Paris policy vs Japan policy
```

If chunking cuts a sentence in half, retrieval may miss the condition:

```
"Refunds are allowed..."
```

but lose:

```
"...only if cancelled 24 hours before arrival."
```

That can cause a dangerous wrong answer.

---

7. Embeddings and indexing

How LlamaIndex uses embeddings

At ingestion time:

```
chunk text → embedding model → vector
```

At query time:

```
user question → same/similar embedding model → query vector
```

Then vector search finds chunks whose vectors are close to the query vector.

Index types at a high level

Do not memorize many index names first. Understand the categories.

Vector index Best for semantic search.

```
Question: "How do I get refund?"
Finds chunks about cancellation, refund, reimbursement.
```

Summary-style index Best when you need summarization over many documents.

```
Question: "Summarize all safety incidents from last month."
```

Keyword/BM25-style retrieval Best when exact words matter.

```
Question: "Policy ID PRK-REF-2026-04"
```

Graph-style index Useful when relationships matter.

```
Attraction → Park → Region → Maintenance Policy → Safety Procedure
```

For most beginner RAG systems, start with a vector index plus good metadata. Then add hybrid retrieval or graph approaches only when needed.

Metadata-aware indexing

In production, you usually store metadata with each node:

```
node_text = "Guests may cancel..."
metadata = {
  "source": "ticket_policy.pdf",
  "department": "Parks",
  "region": "US",
  "policy_version": "2026-07",
  "access_level": "support_agent",
  "effective_date": "2026-07-01"
}
```

Then at query time:

```
Retrieve only where:
region = user's region
access_level <= user's permission
effective_date <= today
```

This prevents wrong or unauthorized answers.

Index update and refresh thinking

Documents change.

Policies change.

APIs change.

Pricing changes.

So production indexing needs:

```
insert new document
update changed document
delete removed document
refresh stale document
avoid duplicate document
track document version
```

LlamaIndex's ingestion/document-management support can use document IDs and hashes to detect duplicates or changed documents. (LlamaIndex Python Documentation)

Cost and performance considerations

Embedding every chunk costs money and time.

Large chunks mean fewer embeddings, but lower retrieval precision.

Small chunks mean better precision, but more vectors, more storage, and sometimes weaker context.

Remote vector DBs add network latency.

Reranking improves quality but adds latency and model cost.

A Staff-level design must balance:

```
quality
latency
cost
freshness
security
maintainability
```

---

8. Retrieval and query flow

Retriever basics

A retriever answers:

```
Which chunks should the LLM read?
```

It does not answer the user directly.

Query engine basics

A query engine answers:

```
Given a user question, retrieve evidence and produce a final answer.
```

Internally:

```
query_engine = retriever + response_synthesizer
```

LlamaIndex exposes both high-level query engine APIs and lower-level composition APIs where you explicitly build a retriever and response synthesizer. (LlamaIndex Python Documentation)

Similarity search

Similarity search means:

```
Find chunks with meaning close to the query.
```

Example:

```
Query:
"Can I change my park booking?"

Similar chunks:
– "Guests may modify reservations..."
– "Booking changes are allowed until..."
– "Reservation cancellation rules..."
```

Metadata filtering

Metadata filtering narrows search using labels.

Example:

```
Question:
"What is the cancellation policy?"

Without filter:
Search all docs globally.

With filter:
Search only:
department = Parks
region = US
policy_status = Active
```

This improves accuracy and access control.

Hybrid retrieval

Hybrid retrieval combines semantic search and keyword search.

Semantic search finds meaning.

Keyword search finds exact terms.

Hybrid is useful when both matter.

Example:

```
Query:
"What does SOP-INC-447 say about ride downtime?"

Keyword search catches:
"SOP-INC-447"

Semantic search catches:
"ride downtime", "attraction outage", "temporary closure"
```

Reranking basics

First retrieval may return 20 chunks.

Reranking chooses the best 5.

```
Retrieve broad
→ rerank precisely
→ send only best context to LLM
```

This often improves answer quality because the LLM receives cleaner evidence.

Response synthesis

The response synthesizer writes the final answer using retrieved chunks.

Production instruction should be strict:

```
Answer only using retrieved context.
If context is missing, say you do not know.
Cite the source.
Do not invent policy.
```

Citation-aware answering

Citation-aware answering means the system shows which source supported the answer.

Example:

```
Answer:
Guests can cancel up to 24 hours before arrival, except for special event
tickets.

Sources:
- ticket_policy_2026.pdf, section 4.2
- special_event_terms.pdf, section 2.1
```

For Disney-like systems, citations are critical because internal teams need to trust the answer.

How retrieval quality affects answer quality

The LLM can only answer well if it receives the right evidence.

```
Wrong retrieved chunks → confident wrong answer
Missing chunks → incomplete answer
Too many chunks → noisy answer
Old chunks → stale answer
Unauthorized chunks → security issue
```

So production RAG debugging starts with retrieval, not the LLM.

---

9. Search and optimization

Chunk size trade-off

Large chunks

Good:

```
More surrounding context
Fewer vectors
Cheaper indexing
```

Bad:

```
May include unrelated text
Less precise retrieval
Higher token cost per answer
```

Small chunks

Good:

```
Precise retrieval
Lower context noise
```

Bad:

```
Can lose important surrounding meaning
More vectors
More storage
More embedding cost
```

Practical starting point:

```
Use medium chunks.
Preserve headings.
Avoid splitting tables badly.
Keep policy clauses together.
Evaluate with real questions.
```

Chunk overlap

Chunk overlap means repeating some text between neighboring chunks.

Example:

```
Chunk 1: lines 1–20
Chunk 2: lines 16–35
```

Why?

Because important meaning often sits near chunk boundaries.

But too much overlap increases storage and cost.

Better metadata design

Good metadata fields:

```
source_id
source_name
document_type
business_unit
region
language
effective_date
expiry_date
version
owner_team
access_level
tenant_id
created_at
updated_at
```

Bad metadata:

```
misc = "some policy thing"
tag = "doc"
```

Good metadata allows precise filtering, security, analytics, debugging, and freshness control.

Query rewriting

Query rewriting means changing the user query into a better search query.

User asks:

```
"Can I get my money back?"
```

Rewrite:

```
"refund cancellation reimbursement policy park ticket"
```

This can improve retrieval.

But be careful. Bad rewriting can change the meaning.

Top-k tuning

Top-k means how many chunks to retrieve.

Example:

```
top_k = 3
```

means retrieve 3 chunks.

Small top-k:

```
fast, cheap, but may miss evidence
```

Large top-k:

```
better recall, but more noise, latency, and token cost
```

Production tuning should use evaluation data, not guessing.

Context quality optimization

Ask:

```
Are retrieved chunks relevant?
Are they current?
Are they allowed for this user?
Are they too long?
Are they duplicate?
Do they contain the answer?
Do they include citations?
```

This is more important than simply increasing context window size.

Latency and cost optimization

Common strategies:

```
Cache frequent answers
Cache embeddings
Cache ingestion transformations
Use metadata filters before reranking
Use cheaper embedding models when acceptable
Use smaller top-k
Use streaming responses
Use async ingestion
Use batch embedding
Use hybrid search only where needed
Use reranking only for complex queries
```

LlamaIndex ingestion pipelines can cache node/transformation combinations, which helps avoid repeated processing for unchanged data. (LlamaIndex Python Documentation)

---

10. Workflows and agents in LlamaIndex

Workflow concepts

Use a workflow when one query requires multiple controlled steps.

Example:

```
Guest asks:
"Why was my refund denied?"

Workflow:
1. Identify user intent
2. Fetch refund policy
3. Fetch booking status from API
4. Check cancellation date
5. Compare against policy
6. Generate explanation
7. Escalate if uncertain
```

This should not be a single simple vector search.

Agent concepts

Use an agent when the system must decide what to do next.

Example:

```
Question:
"Prepare a report on guest complaints about ride downtime last week."

Agent may decide:
1. Search support tickets
2. Query incident database
3. Retrieve ride operation docs
4. Summarize trends
5. Generate report
```

But agents are less predictable, so for high-risk business logic, prefer deterministic workflows.

Workflows vs simple retrieval

Use **simple retrieval** when:

```
Question → retrieve docs → answer
```

Use **workflow** when:

```
Question → classify → retrieve → call API → validate → answer
```

Use **agent** when:

```
Question → LLM decides which tools/steps are needed
```

Staff-level rule:

```
Use deterministic logic where the business process is known.
Use agents only where flexibility is truly needed.
```

---

11. Production-grade challenges

1. Parsing quality issues

PDFs, tables, slides, scanned documents, and screenshots may parse badly.

Impact:

```
Bad text → bad chunks → bad retrieval
```

Fix:

```
Use better parsers
Validate parsed output
Preserve tables/headings
Add document parsing tests
```

---

2. Inconsistent metadata

One document says:

```
region = "US"
```

Another says:

```
country = "USA"
```

Another says:

```
market = "America"
```

Now filters break.

Fix:

```
Define metadata schema
Validate required fields
Normalize values
Reject bad ingestion records
```

---

3. Retrieval drift

Retrieval drift means retrieval quality changes over time.

Causes:

```
New documents added
Old documents not removed
Embedding model changed
Metadata changed
User query patterns changed
```

Fix:

```
Run retrieval evaluation regularly
Track hit rate and MRR
Monitor top retrieved sources
Keep golden test queries
```

LlamaIndex supports response and retrieval evaluation, including metrics like hit-rate, precision, and mean reciprocal rank for retrievers. (LlamaIndex Python Documentation)

---

4. Freshness issues

The system may answer from an old policy.

Fix:

```
Store effective_date
Store expiry_date
Store version
Filter only active docs
Schedule re-indexing
Use document hashes
```

---

5. Slow queries

Causes:

```
Large vector DB
No metadata pre-filter
Large top-k
Slow reranker
Too much context
Remote network latency
```

Fix:

```
Filter first
Retrieve fewer chunks
Add caching
Use faster vector store
Use async calls
Use streaming
```

---

6. High token cost

If every answer sends 20 large chunks to the LLM, cost increases.

Fix:

```
Use smaller top-k
Compress context
Deduplicate chunks
Summarize long documents
Use cheaper model for simple answers
Use stronger model only for hard queries
```

---

7. Large document collections

At Disney scale, you may have millions of chunks.

Challenges:

```
Index size
Embedding cost
Update frequency
Tenant isolation
Search latency
Access control
```

Fix:

```
Partition by tenant/business unit
Use metadata filters
Use scalable vector DB
Use batch ingestion
Use incremental indexing
Monitor index growth
```

---

8. Multi-tenant isolation

If the same platform serves multiple teams, one team must not see another team's data.

Example:

```
Disney Parks support should not retrieve unreleased Disney+ content
strategy docs.
```

Fix:

```
tenant_id metadata
access_level metadata
permission-aware retriever
server-side filters
authorization checks before retrieval
authorization checks after retrieval
audit logs
```

---

9. Evaluation gaps

Many teams test only the final answer.

That is not enough.

You need to test:

```
Did retrieval find the right chunk?
Did reranking improve ordering?
Did the answer stay faithful to context?
Were citations correct?
Did the system refuse when context was missing?
```

---

10. Observability gaps

You need logs/traces for:

```
user query
rewritten query
metadata filters
retrieved node IDs
similarity scores
reranker scores
final context sent to LLM
LLM response
latency
cost
errors
```

Without this, debugging RAG is guesswork.

---

12. Optimization strategies

Better ingestion design

Do:

```
Parse documents carefully
Preserve headings and sections
Remove headers/footers/noise
Keep tables understandable
Use stable document IDs
Track document hashes
Support incremental updates
```

Do not:

```
Dump raw PDFs directly and hope retrieval works
```

Better metadata design

Create a required metadata schema.

Example:

```
{
  "tenant_id": "parks_us",
  "business_unit": "Parks",
  "region": "US",
  "document_type": "policy",
  "policy_area": "refund",
  "version": "2026-07",
  "effective_date": "2026-07-01",
  "access_level": "support_agent"
}
```

Validate this during ingestion.

Better indexing strategy

Start simple:

```
Vector index + metadata
```

Then improve:

```
Add hybrid search for exact IDs/codes
Add reranking for precision
Add summary index for document-level summaries
Add graph only when relationships matter
```

Do not over-engineer on day one.

Better filtering strategy

Use filters before retrieval where possible.

Example:

```
tenant_id = current tenant
region = user's region
document_status = active
access_level <= user permission
```

This improves:

```
accuracy
latency
security
cost
```

Better retriever setup

Tune:

```
top_k
similarity threshold
hybrid weights
reranker model
metadata filters
query rewriting
```

Evaluate each change.

Do not tune by feeling.

Better synthesis strategy

Prompt the answer generator clearly:

```
Use only the provided context.
Cite sources.
If the answer is missing, say you do not know.
Mention policy exceptions.
Do not make legal/financial promises.
```

For Disney-like systems, this is important because guest-facing or employee-facing answers can create operational risk.

Better evaluation approach

Maintain a golden dataset:

```
Question
Expected source chunk
Expected answer
Expected citation
Allowed policy version
```

Measure:

```
retrieval hit rate
MRR
context precision
context recall
faithfulness
answer relevance
citation accuracy
latency
cost
```

LlamaIndex's evaluation docs separate response evaluation from retrieval evaluation, which is exactly how production RAG should be tested. (LlamaIndex Python Documentation)

Better caching approach

Cache at multiple levels:

```
Parsed document cache
Embedding cache
Ingestion transformation cache
Retriever result cache
Answer cache for common questions
```

But be careful with freshness and permissions.

Never serve cached answers across tenants unless access rules are identical.

When to combine LlamaIndex with other tools

Good production combination:

```
LlamaIndex:
Data ingestion, indexing, retrieval, query engines, evaluation

FastAPI:
Backend API layer

Qdrant / Pinecone / pgvector / OpenSearch:
Vector storage

Postgres:
Application data, document registry, metadata, audit records

Redis:
Caching and rate limiting

Kafka / Celery:
Async ingestion jobs

LangGraph:
Complex stateful workflows

LangChain:
Model/tool integration where useful

Kubernetes:
Deployment and scaling

OpenTelemetry / LangSmith / custom tracing:
Observability
```

---

13. Easy real-world example

Let's build a Disney internal assistant for support agents.

Use case

A support agent asks:

```
"A guest cancelled their park reservation 3 hours before arrival.
Can we offer a refund?"
```

Data sources

```
Refund policy PDF
Park reservation rules
Special event ticket rules
Guest support escalation SOP
Regional policy database
```

LlamaIndex ingestion

```
Load documents
→ parse PDFs
→ split by policy sections
→ attach metadata:
   region
   policy_type
   effective_date
   access_level
   source
→ create embeddings
→ store in vector DB
```

Query flow

```
User asks question
→ classify intent = refund_policy
→ apply filters:
   region = US
   policy_type = refund
   status = active
→ retrieve top 10 chunks
→ rerank top 5
→ synthesize answer
→ cite sources
```

Good answer

```
Based on the active US park reservation policy, a same-day cancellation 3
hours before arrival is normally not eligible for a standard refund.
However, the support agent should check whether the booking falls under an
exception such as documented emergency, weather-related closure, or system
error. Source: refund policy section 4.2 and escalation SOP section 2.1.
```

Bad answer

```
Yes, the guest can get a refund.
```

Why bad?

It has no condition, no source, no policy version, and may be legally/operationally wrong.

---

14. Staff-level interview angle

How to explain LlamaIndex in a system design interview

Say this:

```
LlamaIndex is the data and retrieval layer I would use to build a
production RAG system. It helps ingest documents, parse them into nodes,
attach metadata, generate embeddings, build indexes, retrieve relevant
context, synthesize grounded answers, and evaluate retrieval/response
quality.
```

Then go deeper:

```
For a Disney-scale system, I would not treat LlamaIndex as magic. I would
design the ingestion pipeline, metadata schema, vector store, retriever
configuration, access-control filters, evaluation dataset, caching,
observability, and deployment separately.
```

That sounds Staff-level.

How to choose LlamaIndex vs building directly

Use LlamaIndex when:

```
We need faster delivery
We need document ingestion abstractions
We need query engines/retrievers
We need evaluation helpers
We need connectors
We want modular RAG components
```

Build directly when:

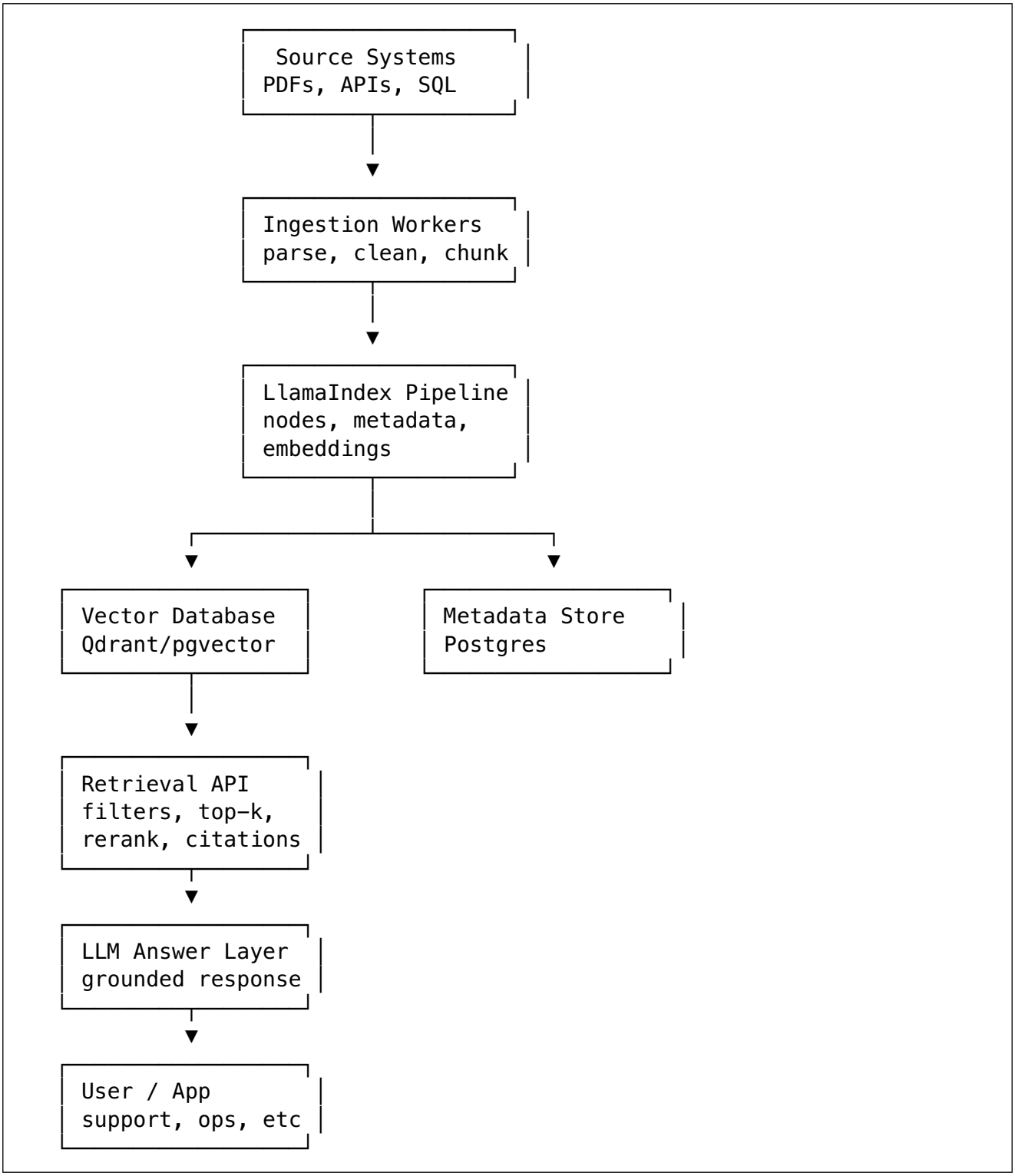
```
The retrieval path is very custom
We need strict control over every storage/query operation
Framework abstraction creates debugging difficulty
The use case is simple enough
```

Staff-level answer:

```
I would start with LlamaIndex for speed and modularity, but keep boundaries
clean so core business logic, authorization, metadata schema, and
evaluation are owned by our platform, not hidden inside the framework.
```

How to use LlamaIndex in production AI backend systems

Production architecture:



Disney-like use cases

LlamaIndex is useful for:

```
Guest support policy assistant
Park operations knowledge search
Legal/compliance document Q&A
Content metadata search
Internal developer runbook assistant
Incident postmortem search
Vendor contract Q&A
HR policy assistant
Knowledge assistant for call-center agents
```

The common pattern is:

```
Large private knowledge base
+ need accurate answers
+ need citations
+ need permission control
+ need freshness
+ need evaluation
```

That is exactly where LlamaIndex becomes valuable.

Final mental model

Remember this one line:

```
LlamaIndex turns messy business data into searchable, retrievable, LLM-
usable knowledge.
```

And this Staff-level chain:

```
Data ingestion
→ clean nodes
→ rich metadata
→ good embeddings
→ correct index
→ precise retrieval
→ grounded synthesis
→ evaluation
→ observability
→ production trust
```

For Day 2, your main takeaway should be:

LlamaIndex is not just "RAG code." It is a production-oriented data framework for building reliable AI systems over private business knowledge.